Introduction to AI

Deterministic Planning and State-space Search

Alex Shleyfman and Gal A. Kaminka

Dept. of Computer Science, Bar-Ilan University

Winter 2026/2027

Work Plan

1. ~~Introduction~~ ~~Uninformed Search~~ (14.10)
2. Heuristics Search (21.10)
3. Domain-Independent Heuristics (28.10)
4. Inference CSP/SAT (04.11) Ex. 1 published
5. Bayesian inference (11.11)
6. Markov Decision Processes (18.11)
7. Multi-Armed Bandits (25.11) Ex. 2 published
8. Reinforcement Learning (02.12)
9. Logic Inference (09.12)
10. Games (16.12) Ex. 3 published
11. Multi-Agent Systems (23.12)
12. MARL (30.12)
13. Plan-Space Planning (06.12)
14. Rehearsal (13.01)

Heuristic search algorithms: systematic

- Heuristic search algorithms are the most common and overall most successful algorithms for deterministic planning.

Popular systematic heuristic search algorithms:

- greedy best-first search
- A*
- weighted A*
- IDA*
- depth-first branch-and-bound search
- breadth-first heuristic search
- ...

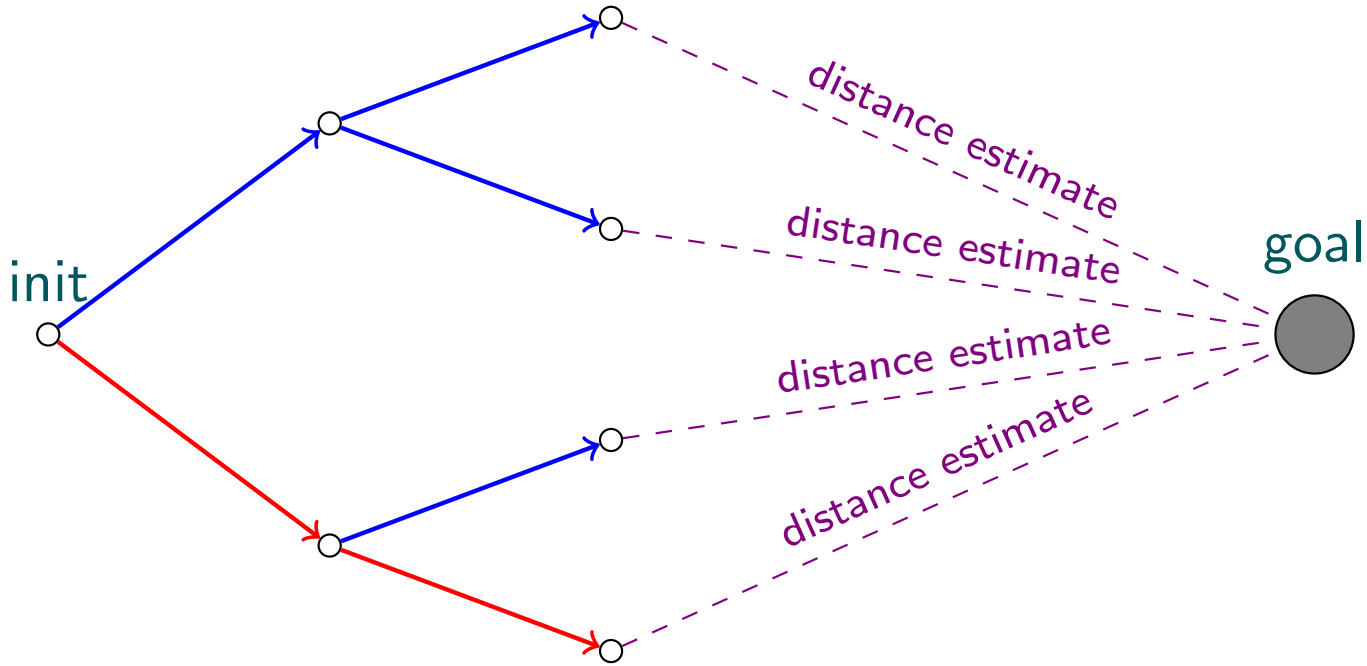
Heuristic search algorithms: local

- Heuristic search algorithms are the most common and overall most successful algorithms for deterministic planning.

Popular heuristic local search algorithms:

- hill-climbing
- enforced hill-climbing
- beam search
- tabu search
- genetic algorithms
- simulated annealing
- ...

Heuristic search: idea



Required ingredients for heuristic search

A heuristic search algorithm requires one more operation in addition to the definition of a search space.

Definition (heuristic function)

Let Σ be the set of nodes of a given search space.
A heuristic function or heuristic (for that search space) is a function $h : \Sigma \to \mathbb{N}_0 \cup \{\infty\}$.

- The value $h(\sigma)$ supposed to estimate the distance from node σ to the nearest goal node.
- Typically: $h(\sigma) \stackrel{\text{def}}{=} h(\mathit{state}(\sigma))$

What exactly is a heuristic estimate?

What does it mean that h "estimates the goal distance"?

- For most heuristic search algorithms, h does not need to have any strong properties for the algorithm to work
- However, the efficiency of the algorithm closely relates to how accurately h reflects the actual goal distance.
- For some algorithms, like A*, we can prove strong formal relationships between properties of h and properties of the algorithm (optimality, dominance, run-time for bounded error, ...)
- For other search algorithms, "it works well in practice" is often as good an analysis as one gets.

Perfect heuristic

Let Σ be the set of nodes of a given search space.

Definition (optimal/perfect heuristic)

The optimal or perfect heuristic of a search space is the heuristic h^* which maps each search node σ to the length of a shortest path from $state(\sigma)$ to any goal state.

Note: $h^*(\sigma) = \infty$ iff no goal state is reachable from σ.

Properties of heuristics

A heuristic h is called

- safe if for all $\sigma \in \Sigma$ s.t $h(\sigma) = \infty$ implies $h^*(\sigma) = \infty$
- goal-aware if $h(\sigma) = 0$ for all goal nodes $\sigma \in \Sigma$
- admissible if $h(\sigma) \leq h^*(\sigma)$ for all nodes $\sigma \in \Sigma$
- consistent if $h(\sigma) \leq h(\sigma') + cost(\sigma, \sigma')$
  for all nodes $\sigma, \sigma' \in \Sigma$ such that σ' is a successor of σ
- blind if for all $\sigma \in \Sigma$ it holds $h(\sigma) = 0$

Relationships?

Greedy best-first search

Greedy best-first search (with duplicate detection)

```
open := new min-heap ordered by (σ ↦ h(σ))
open.insert(make-root-node(init()))
closed := ∅
while not open.empty():
    σ = open.pop-min()
    if state(σ) ∉ closed:
        closed := closed ∪ {state(σ)}
        if is-goal(state(σ)):
            return extract-solution(σ)
        for each ⟨o, s⟩ ∈ succ(state(σ)):
            σ' := make-node(σ, o, s)
            if h(σ') < ∞:
                open.insert(σ')
return unsolvable
```

Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

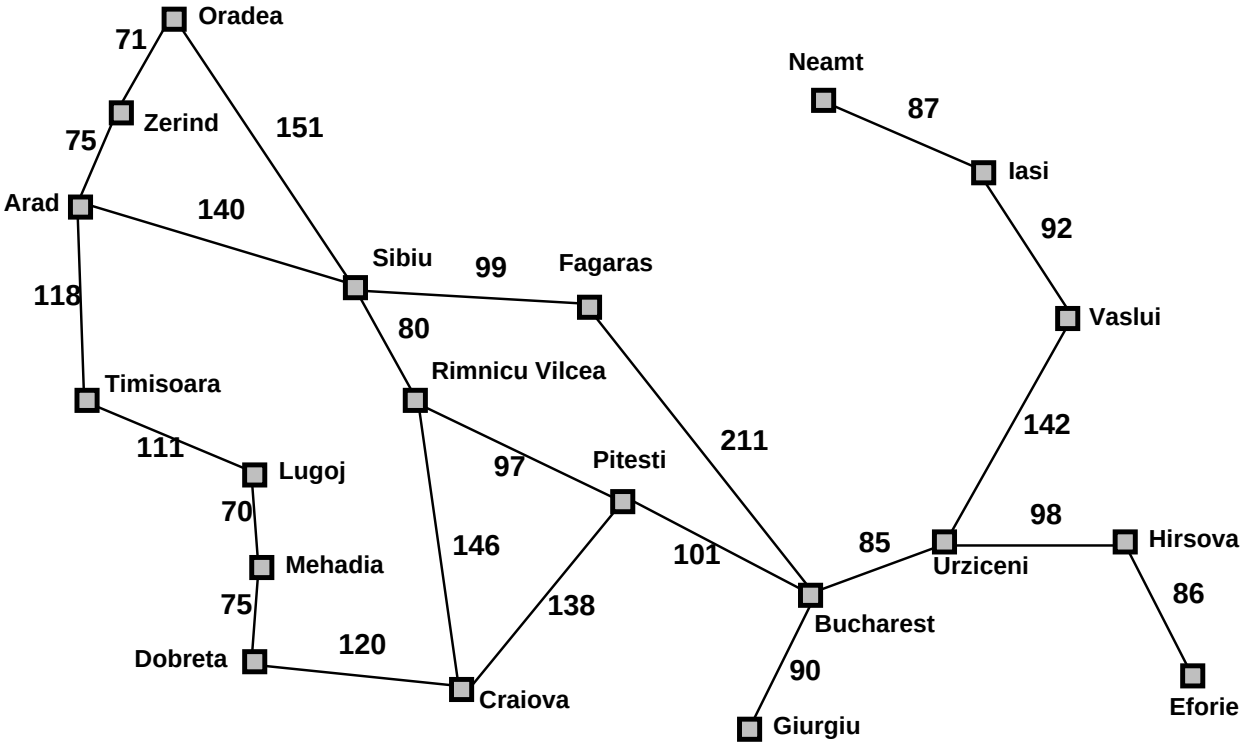
Heuristic functions

Systematic search

Local search

Heuristics

Planning trip to Bucharest with SLD heuristic



Straight−line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

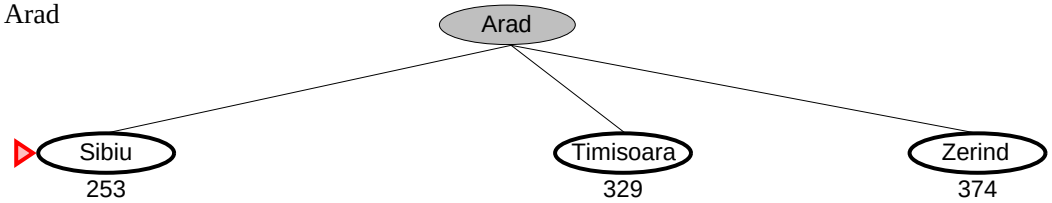
Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Heuristic functions
Systematic search
Local search
Heuristics

GBFS by example

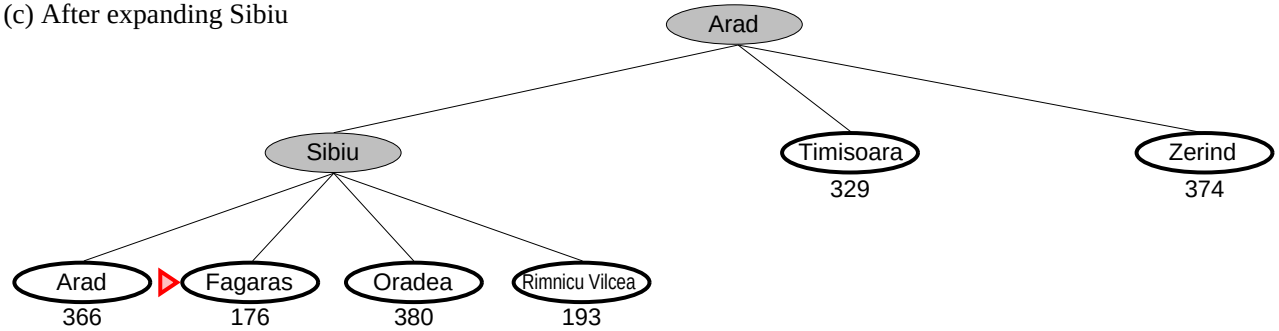
(a) The initial state



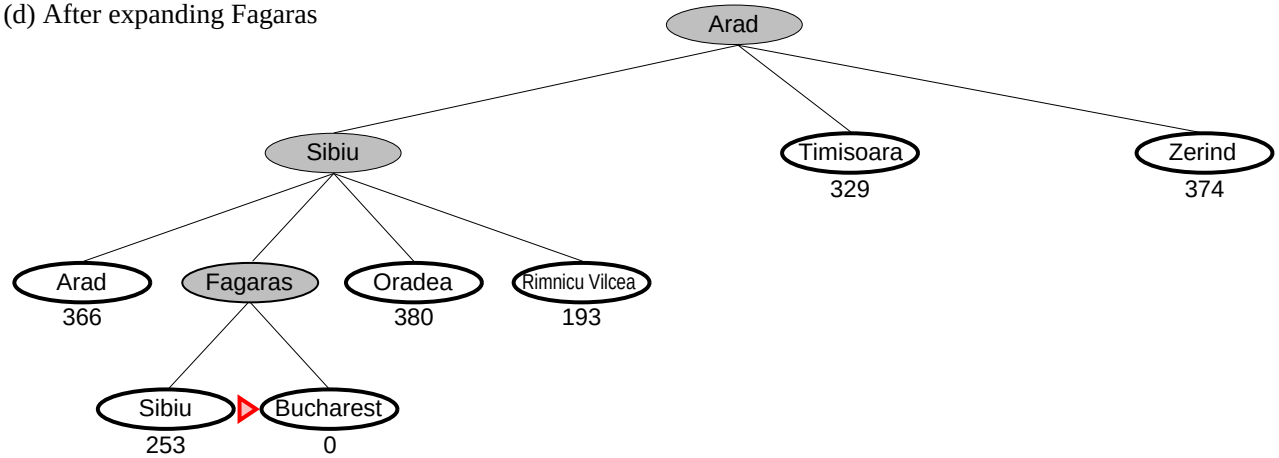
(b) After expanding Arad



(c) After expanding Sibiu



(d) After expanding Fagaras



Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Heuristic functions
Systematic search
Local search
Heuristics

Properties of greedy best-first search

- one of the three most commonly used algorithms for satisficing planning
- complete for safe heuristics (due to duplicate detection)
- suboptimal unless h satisfies some very strong assumptions (similar to being perfect)
- invariant under all strictly monotonic transformations of h (e. g., scaling with a positive constant or adding a constant)

A*

A* (with duplicate detection and reopening)

```
open := new min-heap ordered by (σ ↦ f(σ) = g(σ) + h(σ))
open.insert(make-root-node(init()))
closed := ∅
distance := ∅
while not open.empty():
    σ = open.pop-min()
    if state(σ) ∉ closed or g(σ) < distance(state(σ)):
        closed := closed ∪ {state(σ)}
        distance(state(σ)) := g(σ)
        if is-goal(state(σ)):
            return extract-solution(σ)
        for each ⟨o, s⟩ ∈ succ(state(σ)):
            σ' := make-node(σ, o, s)
            if h(σ') < ∞:
                open.insert(σ')
return unsolvable
```

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Heuristic functions
Systematic search
Local search
Heuristics

A*(one node maintained per state)

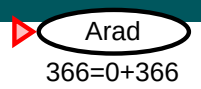
A* (with duplicate detection and reopening)

open := **new** min-set-heap ordered by $(\sigma \mapsto f(\sigma) = g(\sigma) + h(\sigma))$
open.insert(make-root-node(init()))
closed := $\emptyset$
distance := $\emptyset$
while not *open*.empty():
    σ = *open*.pop-min()
    if *state*(σ) $\notin$ *closed* **or** $g(\sigma) <$ *distance*(*state*(σ)):
        closed := *closed* $\cup \{$*state*(σ)$\}$
        distance(*state*(σ)) := $g(\sigma)$
        if is-goal(state(σ)):
            return extract-solution(σ)
        for each $\langle o, s \rangle \in$ succ(*state*(σ)):
            σ' := make-node(σ, o, s)
            if $h(\sigma') < \infty$:
                open.insert(σ')
return unsolvable

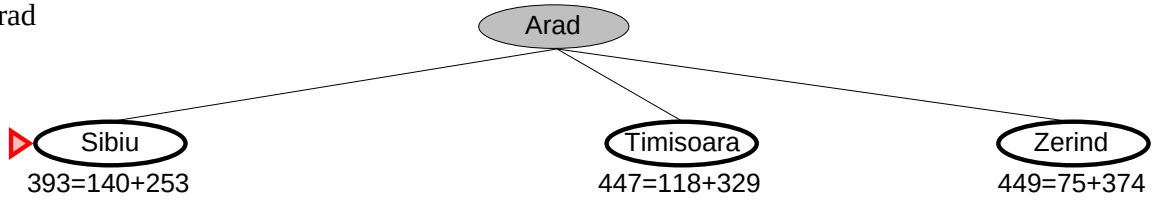
Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Heuristic functions
Systematic search
Local search
Heuristics

A* by example

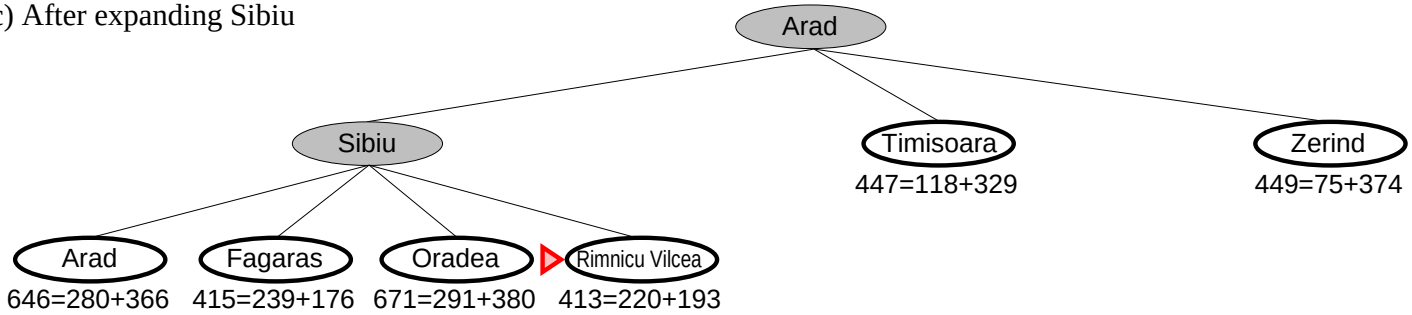
(a) The initial state



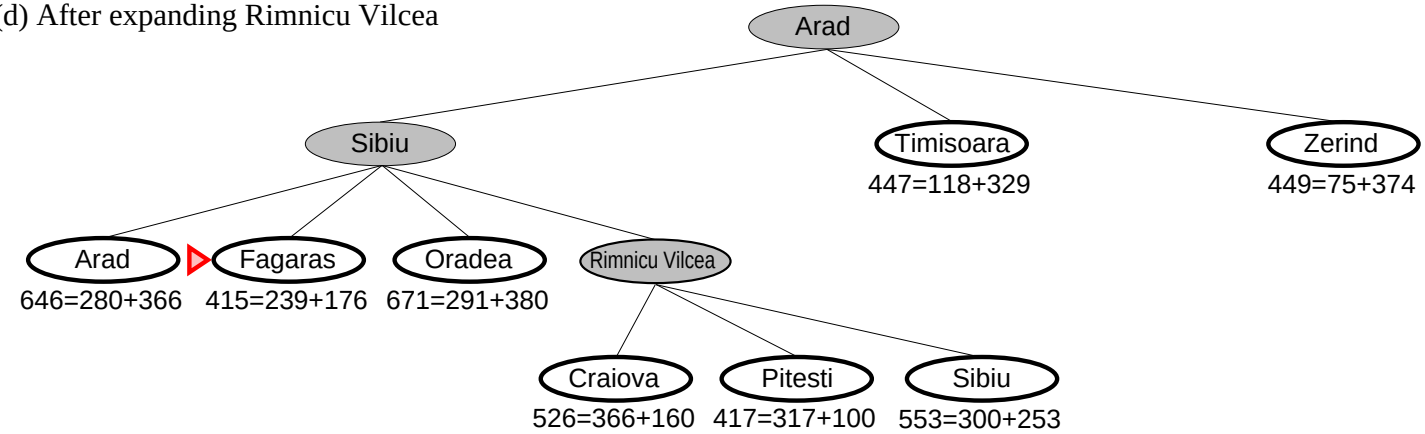
(b) After expanding Arad



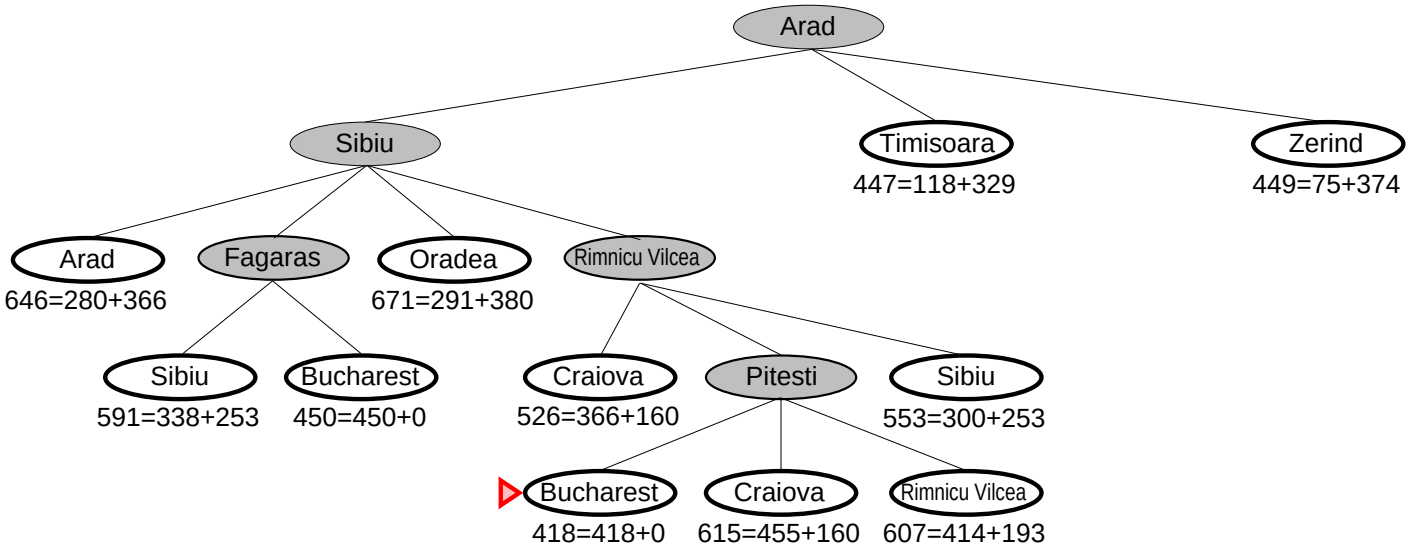
(c) After expanding Sibiu



(d) After expanding Rimnicu Vilcea



A* by example



Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Heuristic functions

Systematic search

Local search

Heuristics

Terminology for A*

- f value of a node – defined by $f(\sigma) := g(\sigma) + h(\sigma)$
- generated nodes – nodes inserted into *open* at some point
- expanded nodes – nodes σ popped from *open* for which the test against *closed* and *distance* succeeds
- reexpanded nodes – expanded nodes for which $\mathit{state}(\sigma) \in \mathit{closed}$ upon expansion (also called reopened nodes)

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Heuristic functions
Systematic search
Local search
Heuristics

Properties of A*

- the most commonly used algorithm for optimal planning
- rarely used for satisficing planning
- complete for safe heuristics
  (even without duplicate detection)
- optimal for admissible heuristics

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Heuristic functions
Systematic search
Local search
Heuristics

Optimality of Tree-Search A* with Admissible h

Standard proof

Suppose to the contrary that the algorithm returns a suboptimal plan via *extract-solution*(σ) for some node σ with *state*$(\sigma) \in G$.

- If so, then no optimal goal node σ^* was in the open list (right?)
- If so, then open list contains some unexpanded node σ' on the (optimal) path from root node to σ^*

$$
\begin{aligned}
f(\sigma) &= g(\sigma) + 0 && \text{since } h(\sigma) = 0 \\
&> g(\sigma^*) && \text{since } \sigma \text{ is suboptimal} \\
&\geq g(\sigma') + h(\sigma') && \text{since } h \text{ is admissible} \\
&= f(\sigma') && \text{a contradiction}
\end{aligned}
$$

$f(\sigma) > f(\sigma') \rightsquigarrow$ contradiction with "σ is dequeued before σ'"

Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Heuristic functions

Systematic search

Local search

Heuristics

Optimality of Tree-Search A* with Admissible h

Standard proof

Suppose to the contrary that the algorithm returns a suboptimal plan via *extract-solution*(σ) for some node σ with *state*$(\sigma) \in G$.

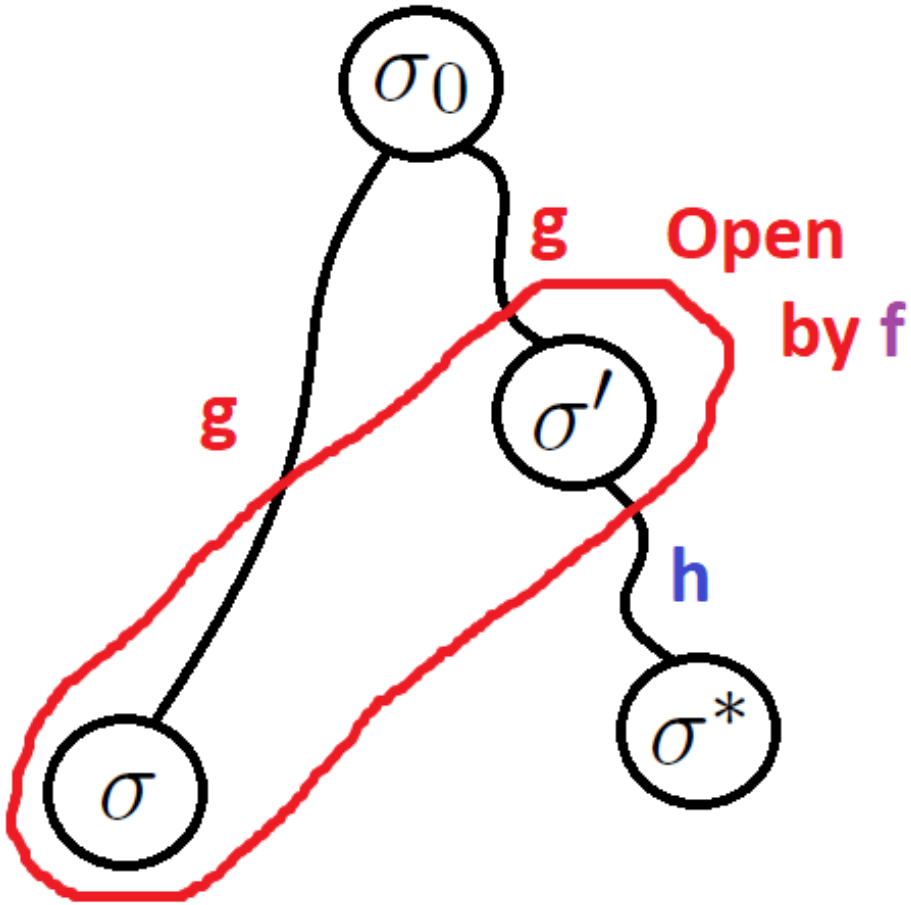
- If so, then no optimal goal node σ^* was in the open list (right?)
- If so, then open list contains some unexpanded node σ' on the (optimal) path from root node to σ^*

$$
\begin{aligned}
f(\sigma) &= g(\sigma) + 0 && \text{since } h(\sigma) = 0 \\
&> g(\sigma^*) && \text{since } \sigma \text{ is suboptimal} \\
&\geq g(\sigma') + h(\sigma') && \text{since } h \text{ is admissible} \\
&= f(\sigma') && \text{a contradiction}
\end{aligned}
$$

$f(\sigma) > f(\sigma') \rightsquigarrow$ contradiction with "σ is dequeued before σ'"

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Heuristic functions
Systematic search
Local search
Heuristics

A* proof by example



Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Heuristic functions
Systematic search
Local search
Heuristics

Weighted A*

Weighted A* (with duplicate detection and reopening)

open := **new** min-heap ordered by $(\sigma \mapsto g(\sigma) + W \cdot h(\sigma))$
open.insert(make-root-node(init()))
closed := $\emptyset$
distance := $\emptyset$
while not *open*.empty():
    σ = *open*.pop-min()
    if *state*(σ) $\notin$ *closed* **or** $g(\sigma) <$ *distance*(*state*(σ)):
        closed := *closed* $\cup \{$*state*(σ)$\}$
        distance(σ) := $g(\sigma)$
        if is-goal(state(σ)):
            return extract-solution(σ)
        for each $\langle o, s \rangle \in$ succ(*state*(σ)):
            σ' := make-node(σ, o, s)
            if $h(\sigma') < \infty$:
                open.insert(σ')
return unsolvable

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Heuristic functions
Systematic search
Local search
Heuristics

Properties of weighted A*

The weight $W \in \mathbb{R}_0^+$ is a parameter of the algorithm.

- for $W = 0$, behaves like breadth-first search
- for $W = 1$, behaves like A*
- for $W \to \infty$, behaves like greedy best-first search

Properties:

- one of the three most commonly used algorithms for satisficing planning
- for $W > 1$, can prove similar properties to A*, replacing optimal with bounded suboptimal: generated solutions are at most a factor W as long as optimal ones

Introduction to AI

Alex Shleyfman and Gal A. Kaminka

Introduction

Heuristic functions

Systematic search

Local search

Heuristics

Hill-climbing

Hill-climbing

$\sigma :=$ make-root-node(init())
forever:
    if is-goal(state(σ)):
        return extract-solution(σ)
    $\Sigma' :=$ { make-node(σ, o, s) | $\langle o, s \rangle \in$ succ(state(σ)) }
    $\sigma :=$ an element of Σ' minimizing h (random tie breaking)

- can easily get stuck in local minima where immediate improvements of $h(\sigma)$ are not possible
- many variations: tie-breaking strategies, restarts

Enforced hill-climbing

Enforced hill-climbing: procedure improve

```
def improve(σ_0):
    queue := new fifo-queue
    queue.push-back(σ_0)
    closed := ∅
    while not queue.empty():
        σ = queue.pop-front()
        if state(σ) ∉ closed:
            closed := closed ∪ {state(σ)}
            if h(σ) < h(σ_0):
                return σ
            for each ⟨o, s⟩ ∈ succ(state(σ)):
                σ' := make-node(σ, o, s)
                queue.push-back(σ')
    fail
```

$\rightsquigarrow$ breadth-first search for more promising node than σ_0

Enforced hill-climbing (ctd.)

Enforced hill-climbing

$\sigma :=$ make-root-node(init())
while not is-goal(state(σ)):
    $\sigma :=$ improve(σ)
return extract-solution(σ)

- one of the three most commonly used algorithms for satisficing planning
- can fail if procedure *improve* fails (when the goal is unreachable from σ_0)
- complete for undirected search spaces (where the successor relation is symmetric) if $h(\sigma) = 0$ for all goal nodes and only for goal nodes

Classification: what works where in (deterministic) planning?

uninformed vs. heuristic search:

- For satisficing planning, heuristic search vastly outperforms uninformed algorithms on most domains.
- For optimal planning, heuristic search typically outperforms uninformed algorithms, but an efficiently implemented uninformed algorithm is not easy to beat in most domains.

systematic search vs. local search:

- For satisficing planning, the most successful algorithms are somewhere between the two extremes.
- For optimal planning, systematic algorithms are required.

Where heuristics come from?

General idea

(Admissible) heuristic functions obtained as
(optimal) cost functions of relaxed problems

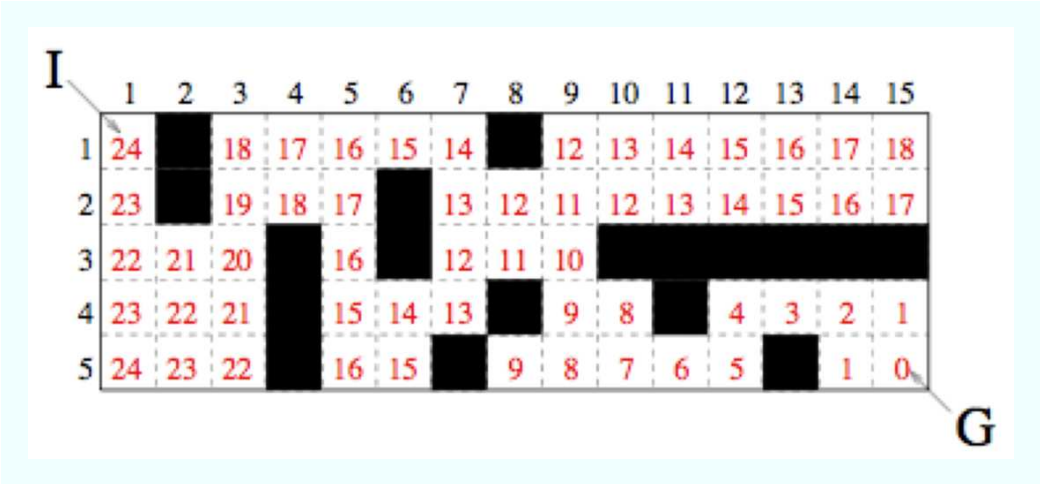
Examples

- Euclidian distance in Path Finding
- Manhattan distance in N-puzzle
- Spanning Tree in Traveling Salesman Problem
- Shortest Path in Job Shop Scheduling

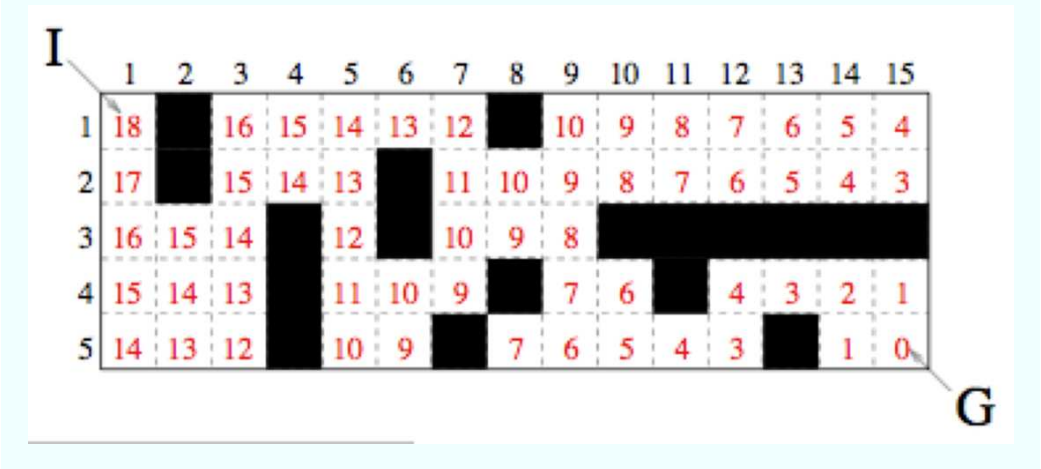
Example

Grid Search and Manhattan Distance

True distance h^* for different search states



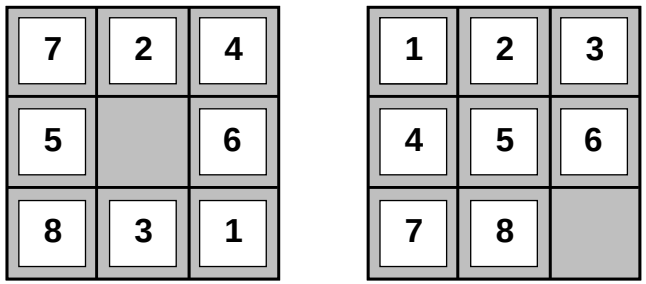
Manhattan distance is based on the relaxation that ...?



Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Heuristic functions
Systematic search
Local search
Heuristics

Example

8-Puzzle



- A tile can move from square A to square B if A is adjacent to B and B is blank $\rightsquigarrow$ solution distance h^*
- A tile can move from square A to square B if A is adjacent to B $\rightsquigarrow$ manhattan distance heuristic h^{MD}
- A tile can move from square A to square B $\rightsquigarrow$ misplaced tiles heuristic h^{MT}

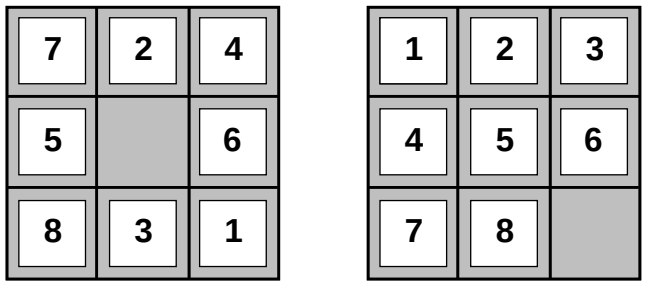
Here: $h^*(s_0) = ?$, $h^{MD}(s_0) = 14$, $h^{MT}(s_0) = 6$

In general, $h^* \geq h^{MD} \geq h^{MT}$. *(Why?)*

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Heuristic functions
Systematic search
Local search
Heuristics

Example

8-Puzzle



- A tile can move from square A to square B if A is adjacent to B and B is blank $\rightsquigarrow$ solution distance h^*
- A tile can move from square A to square B if A is adjacent to B $\rightsquigarrow$ manhattan distance heuristic h^{MD}
- A tile can move from square A to square B $\rightsquigarrow$ misplaced tiles heuristic h^{MT}

Here: $h^*(s_0) = ?$, $h^{MD}(s_0) = 14$, $h^{MT}(s_0) = 6$
In general, $h^* \geq h^{MD} \geq h^{MT}$. *(Why?)*

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Heuristic functions
Systematic search
Local search
Heuristics

Dominance relation between admissible heuristics

Precision matters

Given two admissible heuristics h_1, h_2, if $h_2(\sigma) \geq h_1(\sigma)$ for all search nodes σ, then h_2 dominates h_1 and is better for optimizing search

Typical search costs (unit-cost action)

$h^*(I) = 14$	BFS $\approx$ 1,700,000 nodes
	A$^*(h_1) \approx$ 560 nodes
	A$^*(h_2) \approx$ 115 nodes
$h^*(I) = 24$	BFS $\approx$ 27,000,000,000 nodes
	A$^*(h_1) \approx$ 40,000 nodes
	A$^*(h_2) \approx$ 1,650 nodes

Dominance relation between admissible heuristics

Precision matters

Given two admissible heuristics h_1, h_2, if $h_2(\sigma) \geq h_1(\sigma)$ for all search nodes σ, then h_2 dominates h_1 and is better for optimizing search

Combining admissible heuristics

For any admissible heuristics $h_1, \ldots, h_k$,

$$h(\sigma) = \max_{i=1}^{k} \{h_i(\sigma)\}$$

is also admissible and dominates all individual h_i

Introduction to AI
Alex Shleyfman and Gal A. Kaminka
Introduction
Heuristic functions
Systematic search
Local search
Heuristics

Are we a solver?

General idea

(Admissible) heuristic functions obtained as (optimal) cost functions of relaxed problems

- OK, but heuristic is yet another input to our agent!
- Satisfactory for general solvers?
- Satisfactory in special purpose solvers?

Towards domain-independent agents

- How to get heuristics automatically?
- Can such automatically derived heuristics dominate the domain-specific heuristics crafted by hand?

Are we a solver?

General idea

(Admissible) heuristic functions obtained as (optimal) cost functions of relaxed problems

- OK, but heuristic is yet another input to our agent!
- Satisfactory for general solvers?
- Satisfactory in special purpose solvers?

Towards domain-independent agents

- How to get heuristics automatically?
- Can such automatically derived heuristics dominate the domain-specific heuristics crafted by hand?